

PKU-Zurich Summer School

Lecture-2: Deep Learning for Macro-Finance

Goutham GOPALAKRISHNA (ROTMAN-UTORONTO)

[based on joint work with]

Zhouzhou GU (Princeton)

Jonathan PAYNE (Princeton)

Winston WU (University of Toronto)

July, 2026

Outline

Applications

- Portfolio Choice Problem

Consumption-Saving Problem

- ... with Poisson Shocks

- ... with Brownian Shocks

Deep Learning for PDEs

- Active Learning

Applications

1. Applications

1.1. Portfolio Choice Problem

Portfolio Choice Problem

Consider an entrepreneur who chooses controls (c_t, θ_t) to solve

$$V_t := \sup_{c_t, \theta_t \in \mathcal{C}(X)} \mathbb{E}_t \left[\int_t^\infty e^{-\rho s} u(c_s) ds \right]$$

subject to the wealth process given by

$$\frac{dX_t}{X_t} = (r - c_t/X_t)dt + \theta_t(\mu - r)dt + \sigma\theta_t dW_t$$

where

- c_t is the consumption,
- θ_t is the portfolio weight,
- r is the risk-free rate,
- μ is the expected return of the stock,
- σ is the volatility of the stock.

Portfolio Choice Problem

Let $V(x)$ denote the value function at time t . Then the HJBE is given by

$$\rho V(x) = \sup_{c, \theta \in \mathcal{C}(x)} \left(u(c) + \frac{\partial V}{\partial x} x (r + \theta(\mu - r) - c/x) + \frac{1}{2} \frac{\partial^2 V}{\partial x^2} \sigma^2 x^2 \theta^2 \right)$$

where $X_t = x$. The FOCs are

$$[c] : \quad u'(c) = V'(x)$$

$$[\theta] : \quad 0 = \left(\frac{\mu - r}{\sigma^2} \right) \left(-\frac{V'(x)}{x V''(x)} \right)$$

Portfolio Choice Problem

Assume $u(c) = \frac{c^{1-\gamma}}{1-\gamma}$ for $\gamma > 0$ and guess the value function of the form

$$V(x) = \psi^{-\gamma} \left(\frac{a^{1-\gamma}}{1-\gamma} \right) \implies V'(x) = \psi^{-\gamma} a^{-\gamma}; \quad V''(x) = -\gamma \psi^{-\gamma} a^{-\gamma-1}$$

The FOCs become

$$[c] : \quad c = \psi x$$

$$[\theta] : \quad \theta = \frac{1}{\gamma} \left(\frac{\mu - r}{\sigma^2} \right)$$

Portfolio Choice Problem

The HJBE becomes:

$$\rho V(x) = \sup_{c, \theta \in \mathcal{C}(x)} \left\{ u(c) + V'(x) ((r + (\mu - r)\theta)x - c) + \frac{1}{2} \sigma^2 \theta^2 x^2 V''(x) \right\}$$

$$\begin{aligned} \Rightarrow \rho \frac{\psi^{-\gamma} x^{1-\gamma}}{1-\gamma} &= \frac{(\psi x)^{1-\gamma}}{1-\gamma} + \psi^{-\gamma} x^{-\gamma} ((r + (\mu - r)\theta)x - \psi x) \\ &\quad - \frac{1}{2} \sigma^2 \theta^2 x^2 \gamma \psi^{-\gamma} x^{-\gamma-1} \\ \Rightarrow \psi &= \frac{1}{\gamma} \left[\rho - (1-\gamma) \left(r + \frac{1}{2\gamma} \left(\frac{\mu - r}{\sigma} \right)^2 \right) \right] \end{aligned}$$

Consumption-Saving Problem

2. Consumption-Saving Problem

2.1. ... with Poisson Shocks

Consumption-Saving Problem

Consider households with wealth x and labor hours y . They solve

$$V_t = \sup_{c, y \in \mathcal{C}(x, y)} \mathbb{E}_t \left[\int_t^\infty e^{-\rho s} u(c_s) ds \right]$$

where

$$dx_t = (r_t x_t + w_t y_t - c_t) dt$$

$$y_t \in \{y_1, y_2\} \text{ (Poisson Process)}$$

$$c_t \geq 0$$

- ρ is the discount rate,
- r_t is the risk-free rate,
- w_t is the wage rate,
- y_t is the labor hours ($y_1 < y_2$) with switching intensities $\{\lambda_1, \lambda_2\}$,
- c_t is the consumption rate.

Consumption-Saving Problem

The HJBE is given by

$$\rho V(x, y) = \sup_{c, y \in \mathcal{C}(x, y)} \{u(c) + \partial_x V(x, y) (rx + wy - c) + \lambda(y) (V(x, y') - V(x, y))\}$$

The HJBE along the transition path to $V(x, y)$ is

$$t \in [0, T) : \quad \rho V(x, y) = \sup_{c, y \in \mathcal{C}(x, y)} \{u(c) + \partial_x V(x, y) (rx + wy - c) + \lambda(y) (V(x, y') - V(x, y)) + \partial_t V(x, y)\}$$

$$t = T : \quad V(x, y) = V_T$$

2. Consumption-Saving Problem

2.2. ... with Brownian Shocks

Consumption-Saving Problem

Consider households with wealth x and labor hours y . They solve

$$V_t = \sup_{c, y \in \mathcal{C}(x, y)} \mathbb{E}_t \left[\int_t^\infty e^{-\rho s} u(c_s) ds \right]$$

where

$$dx_t = (r_t x_t + w_t y_t - c_t) dt$$

$$dy_t = \mu(y_t) dt + \sigma(y_t) dW_t$$

$$c_t \geq 0$$

where

- ρ is the discount rate,
- r_t is the risk-free rate,
- w_t is the wage rate,
- y_t is the labor hours with $y_1 < y_2$ with switching intensities $\{\lambda_1, \lambda_2\}$,
- c_t is the consumption rate.

HJBE

The HJBE is given by

$$\begin{aligned}\rho V(x, y) = \sup_{c, y \in \mathcal{C}(x, y)} & \left\{ u(c) + \partial_x V(x, y) (rx + wy - c) \right. \\ & \left. + \mu(y) \partial_y V(x, y) + \frac{1}{2} \sigma^2(y) \partial_y^2 V(x, y) \right\}\end{aligned}$$

The HJBE along the transition path to $V(x, y)$ is

$$\begin{aligned}t \in [0, T) : \quad & \rho V(x, y) = \sup_{c, y \in \mathcal{C}(x, y)} \left\{ u(c) + \partial_x V(x, y) (rx + wy - c) \right. \\ & \left. + \mu(y) \partial_y V(x, y) + \frac{1}{2} \sigma^2(y) \partial_y^2 V(x, y) + \partial_t V(x, y) \right\}\end{aligned}$$

$$t = T : \quad V(x, y) = V_T$$

Deep Learning for PDEs

General setup

Consider a household that solves

$$V_t = \sup_{c, y \in \mathcal{C}(x, y)} \mathbb{E}_t \left[\int_0^\infty f(c_s, V_s) ds \right]$$

such that

$$\frac{dw_t}{w_t} = (r_t + \theta_t \zeta_t - c_t/w_t) dt + \theta_t \sigma_t^R dW_t$$

where

- r_t is the risk-free rate,
- θ_t is the portfolio weight in risk asset that is claim to endowment
 $dy_t = y_t(\mu^y dt + \sigma^y dW_t)$
- ζ_t is the expected return of the stock,
- σ_t^R is the volatility of the stock.

If μ_y, σ_y, r are time-varying, then it is a multi-dimensional problem.

General Setup

- Many examples:
 1. Portfolio Choice: Merton (1971), Cochrane et al. (2008), Martin (2013).
 2. Macro-Finance: He-Krishnamurthy (2013), Brunnermeier-Sannikov (2014), Di Tella (2017).
 3. Monetary Models: Brunnermeier-Sannikov (2016), Brunnermeier-Merkel-Sannikov (2025), Alexandrov-Brunnermeier (2026).

HJBE

The HJBE for this general set up is given by

$$0 = \sup_{c, y \in \mathcal{C}(x, y)} \left\{ f(c, V_t) + E_t[dV_t] \right\}$$

Conjecturing that $V_t = \frac{J_t w_t^{1-\gamma}}{1-\gamma}$ for $\gamma > 0$, where J_t is the opportunity process given by

$$\frac{dJ_t}{J_t} = \mu_t^J dt + \sigma_t^J dW_t$$

The HJB equation reduces to

$$\mu^J(x, J)J = \sum_{i=1}^d \mu^{x_i}(x, J) \frac{\partial J}{\partial x_i} + \frac{1}{2} \sum_{i,j=1}^d b_{i,j}(x, J) \frac{\partial^2 J}{\partial x_i \partial x_j}.$$

where μ^J , μ^x , and $b_{i,j}$ are linear, advection, and diffusion coefficients, and x is the state vector.

HJB

Prior literature converts this non-linear PDE into a quasi-linear parabolic PDE and uses finite differences, slowly introducing non-linearity through

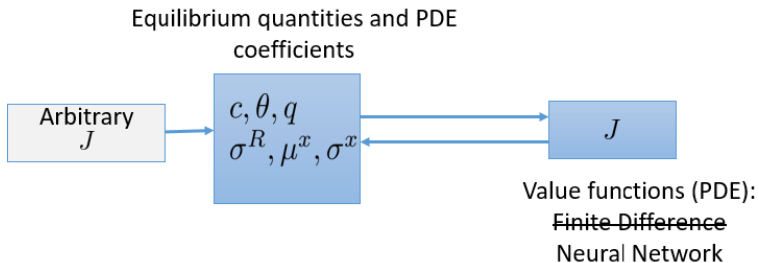
$$\mu^J(x, J^{old})J = \frac{\partial J}{\partial t} + \sum_{i=1}^d \mu^{x_i}(x, J^{old}) \frac{\partial J}{\partial x_i} + \sum_{i,j=1}^d b_{i,j}(x, J^{old}) \frac{\partial^2 J}{\partial x_i \partial x_j}.$$

This works well in low dimensions, but breaks down in high dimensions.

This Lecture

Focus of this part is to introduce a technique to solve macro models involving PDEs of the previous type in high dimensions.

1. Benchmark model: Brunnermeier and Sannikov (2016) with recursive preference.
2. 2D macro-finance models (Di Tella (2017), Gopalakrishna (2021) etc.)



Neural Network Solution Method

$$f := \frac{\partial \hat{f}}{\partial t} + \sum_{i=1}^d \mu^i(x) \frac{\partial \hat{f}}{\partial x_i} + \sum_{i,j=1}^d b^{i,j}(x) \frac{\partial^2 \hat{f}}{\partial x_i \partial x_j} - \mu^J \hat{f} = 0$$

$$\forall (t, x) \in [T - k\Delta t, T - (k - 1)\Delta t] \times \Omega$$

$$\hat{f} = \tilde{f}_0 \quad \forall (t, x) \in (T - (k - 1)\Delta t) \times \Omega.$$

where $\hat{f}$ is a neural network object with parameters Θ , and f is the PDE residual.

Neural Network Solution Method

$$f := \frac{\partial \hat{f}}{\partial t} + \sum_{i=1}^d \mu^i(x) \frac{\partial \hat{f}}{\partial x_i} + \sum_{i,j=1}^d b^{i,j}(x) \frac{\partial^2 \hat{f}}{\partial x_i \partial x_j} - \mu^J \hat{f} = 0$$

Can be seen as a classical constrained optimization problem:

$$\begin{aligned} \Theta^\star &= \arg \min_{\Theta} \left\| \hat{f} - \tilde{f}_0 \right\| \\ \text{s.t. } f &= 0. \end{aligned}$$

Neural Network Solution Method

$$f := \frac{\partial \hat{f}}{\partial t} + \sum_{i=1}^d \mu^i(x) \frac{\partial \hat{f}}{\partial x_i} + \sum_{i,j=1}^d b^{i,j}(x) \frac{\partial^2 \hat{f}}{\partial x_i \partial x_j} - \mu^J \hat{f} = 0$$

Can be seen as a classical constrained optimization problem:

$$\begin{aligned} \Theta^\star &= \arg \min_{\Theta} \left\| \hat{f} - \tilde{J}_0 \right\| \\ \text{s.t.} \quad & \int_t \int_x |f|^2 dt dx = 0. \end{aligned}$$

Neural Network Solution Method

$$f = 0 \quad \forall (t, x) \in [T - k\Delta t, T - (k - 1)\Delta t] \times \Omega,$$

$$\hat{f} = \tilde{J}_0 \quad \forall (t, x) \in (T - (k - 1)\Delta t) \times \Omega,$$

$$\frac{\partial \hat{f}}{\partial x} = J_0 \quad \forall (t, x) \in (T - (k - 1)\Delta t) \times \partial\Omega.$$

- Mesh free since we can randomly sample from the state space (t, x) to train the NN
 $\implies$ Alleviates the curse of dimensionality

Neural Network Solution Method

$$f := \frac{\partial \hat{f}(x|\Theta)}{\partial t} + \sum_i \mu^i(x) \frac{\partial \hat{f}(x|\Theta)}{\partial x_i} + \sum_{i,j=1}^d b^{i,j}(x) \frac{\partial^2 \hat{f}(x|\Theta)}{\partial x_i \partial x_j} - \mu^J \hat{f}(x|\Theta) = 0.$$

$$\hat{f}(x|\Theta) = \tilde{f}_0, \quad (t, x) \in (T - (k-1)\Delta t) \times \Omega,$$

$$f = 0, \quad (t, x) \in [T - k\Delta t, T - (k-1)\Delta t] \times \Omega_c,$$

$$\hat{f}(x|\Theta) = \tilde{f}_0, \quad (t, x) \in (T - (k-1)\Delta t) \times \Omega_c, \quad \rightarrow \text{Active learning},$$

$$\frac{\partial \hat{f}(x|\Theta)}{\partial x} = J_0, \quad (t, x) \in (T - (k-1)\Delta t) \times \partial\Omega.$$

3. Deep Learning for PDEs

3.1. Active Learning

Active Learning

Sparse training points in regions of importance lead to instability in future iterations.

- Residual based
- Adaptive weights
- Attention-based

Global solution methods are prone to instability and NN methods are not an exception. Time-stepping helps with stability.

Active Learning for Multi Loss

Say we have the following aggregate loss function at time (epoch) iteration t :

$$L(t) = \sum_{i=1}^k \lambda_i(t) L_i(t)$$

Note that we let the weight $\lambda_i(t)$ be dependent on time.

Adaptive weight is given by the following softmax function

$$\lambda_i(t) = \frac{\exp(\mathbb{T}(L_i(t) - L_i(t-1)))}{\sum_{j=1}^k \exp(\mathbb{T}(L_j(t) - L_j(t-1)))}$$

where the parameter $\mathbb{T}$ controls the adaptation strength: $\mathbb{T} \rightarrow \infty$ sets uniform weight.

Active Learning with Time Stepping

- Global solution techniques are prone to instabilities.
- Deep neural nets are also good at finding near equilibrium solutions.
- One way to ensure stability and find the equilibrium solution is to introduce time stepping scheme.

$$\text{HJB: } \mu_i^J(x, \hat{J}_i(\Theta_{J_i}))\hat{J}_i(\Theta_{J_i}) = \sum_{j=1}^d \mu^{xj}(x, \hat{J}_i(\Theta_{J_i})) \frac{\partial \hat{J}_i(\Theta_{J_i})}{\partial x_j} + \sum_{j,k=1}^d b^{j,k} \frac{\partial^2 \hat{J}_i(\Theta_{J_i})}{\partial x_j \partial x_k}$$

$$\text{HJB with time step: } \mu_i^J(x, \tilde{J}_i)\hat{J}_i(\Theta_{J_i}) = \frac{\partial \hat{J}_i}{\partial t} + \sum_{j=1}^d \mu^{xj}(x, \tilde{J}_i) \frac{\partial \hat{J}_i(\Theta_{J_i})}{\partial x_j} + \sum_{j,k=1}^d b^{j,k} \frac{\partial^2 \hat{J}_i(\Theta_{J_i})}{\partial x_j \partial x_k}$$

where $\tilde{J}_i$ is known from previous iteration, with an initial guess at the 0-th iteration.

- Time stepping slowly introduces non-linearity.
- Starting with a good guess leads to convergence to correct equilibrium.
- Behaves more like a contraction mapping with rich theoretical properties.

Automatic Differentiation in Practice

Listing 1: Approximating J using a neural network

```
def J(z, t):  
    x = torch.cat([z, t], dim=1)  
    return neural_net(x, weights, biases)
```

Listing 2: Constructing regularizer: 1D model

```
def D(y, x):  
    return torch.autograd.grad(  
        y, x, grad_outputs=torch.ones_like(y), create_graph=True  
    )[0]  
  
def f(z, t):  
    J_val = J(z, t)  
    J_t = D(J_val, t)  
    J_z = D(J_val, z)  
    J_zz = D(J_z, z)  
    return J_t + advection * J_z + diffusion * J_zz - linearTerm * J_val
```

Automatic Differentiation in Practice

Listing 1: Approximating J using a neural network

```
def J(z, t):  
    x = torch.cat([z, t], dim=1)  
    return neural_net(x, weights, biases)
```

Listing 2: Constructing regularizer: 1D model

```
def D(y, x):  
    return torch.autograd.grad(  
        y, x, grad_outputs=torch.ones_like(y), create_graph=True  
    )[0]  
  
def f(z, t):  
    J_val = J(z, t)  
    J_t = D(J_val, t)  
    J_z = D(J_val, z)  
    J_zz = D(J_z, z)  
    return J_t + advection * J_z + diffusion * J_zz - linearTerm * J_val
```

Listing 3: Constructing regularizer: 2D model

```
def f(z, a, t):  
    J_val = J(z, a, t)  
    J_t, J_z, J_a = D(J_val, t), D(J_val, z), D(J_val, a)  
    J_zz, J_aa, J_az = D(J_z, z), D(J_a, a), D(J_a, z)  
  
    return (J_t + advection_z * J_z + advection_a * J_a  
            + diffusion_z * J_zz + diffusion_a * J_aa + crossTerm * J_az)
```

References (i)

- ▶ Shreve, S. E. (2004). *Stochastic Calculus for Finance II: Continuous-Time Models*. Springer Finance. New York: Springer.
- ▶ Karatzas, I. and S. E. Shreve (1998). *Methods of Mathematical Finance*. Vol. 39. Applications of Mathematics. New York: Springer.
- ▶ Øksendal, B. (2003). *Stochastic Differential Equations: An Introduction with Applications*. 6th ed. Berlin: Springer.
- ▶ Applebaum, D. (2009). *Lévy Processes and Stochastic Calculus*. 2nd ed. Vol. 116. Cambridge Studies in Advanced Mathematics. Cambridge: Cambridge University Press.
- ▶ Björk, T. (2012). *Arbitrage Theory in Continuous Time*. 3rd ed. Oxford: Oxford University Press.
- ▶ Pham, H. (2009). *Continuous-Time Stochastic Control and Optimization with Financial Applications*. Vol. 61. Stochastic Modelling and Applied Probability. Berlin: Springer.